

الكلية الجامعية للعلوم التطبيقية

رائدة الإبداع



Project: Analysis of a Suspicious Code Sample and Building a Detection Rule

Instructor: Eng. Samih Salem

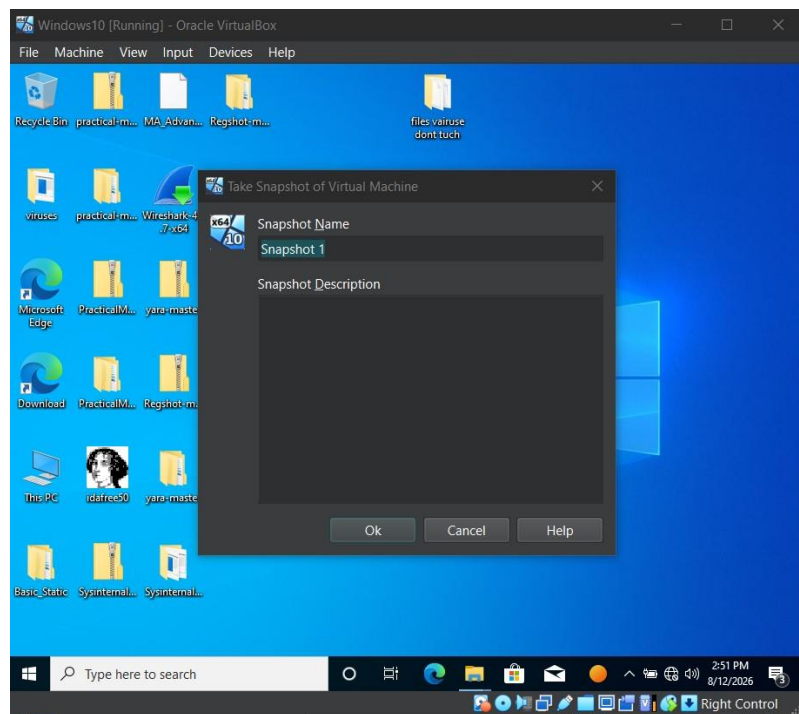
By: Mahmoud Haitham Altawal

ID: 120243939

1. Lab Environment Setup & Safety

The analysis was conducted inside an isolated Oracle VirtualBox Windows 10 VM. The following preparations were made before executing the sample:

Element	Details
Operating System	Windows 10 (x64) inside Oracle VirtualBox
Network Mode	Host-only / Internal Network (No Bridged Mode per instructions)
Snapshot	Snapshot 1 taken before execution for rollback capability
Static Tools	CFF Explorer, PEiD, Strings (Sysinternals), Command Prompt, certutil
Dynamic Tools	Regshot, Process Monitor (Procmon), Process Explorer, Wireshark
Sample	MA_Advanced_Lab_Sample.exe (~5.16 MB)

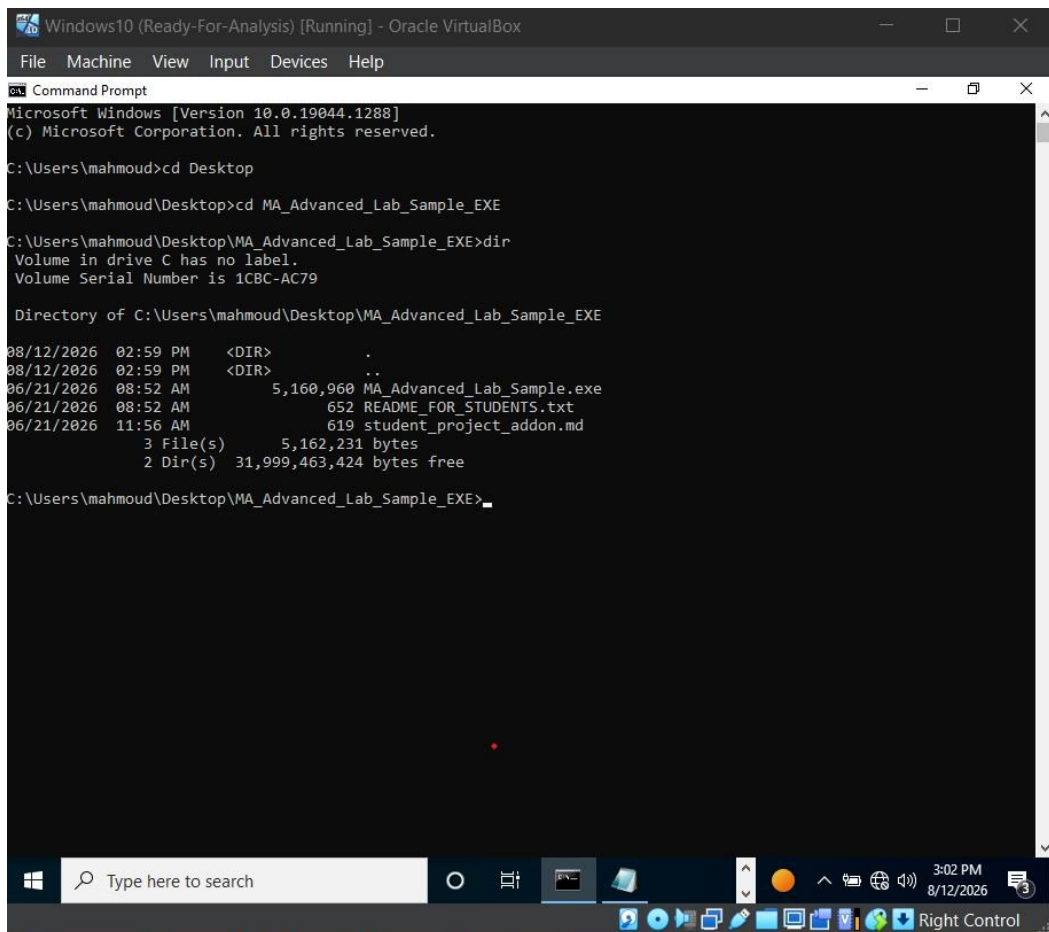


2. Static Analysis

Static analysis was performed without executing the sample. The goal was to extract file metadata, hashes, PE structure, imports, and strings.

2.1 Sample Directory Contents

The sample folder contains three files: the executable, a README for students, and a project addon markdown file.



```
Windows10 (Ready-For-Analysis) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Command Prompt
Microsoft Windows [Version 10.0.19044.1288]
(c) Microsoft Corporation. All rights reserved.

C:\Users\mahmoud>cd Desktop

C:\Users\mahmoud\Desktop>cd MA_Advanced_Lab_Sample_EXE

C:\Users\mahmoud\Desktop\MA_Advanced_Lab_Sample_EXE>dir
Volume in drive C has no label.
Volume Serial Number is 1CBC-AC79

Directory of C:\Users\mahmoud\Desktop\MA_Advanced_Lab_Sample_EXE

08/12/2026  02:59 PM    <DIR>          .
08/12/2026  02:59 PM    <DIR>          ..
06/21/2026  08:52 AM             5,160,960 MA_Advanced_Lab_Sample.exe
06/21/2026  08:52 AM                652 README_FOR_STUDENTS.txt
06/21/2026  11:56 AM                619 student_project_addon.md
               3 File(s)          5,162,231 bytes
               2 Dir(s)  31,999,463,424 bytes free

C:\Users\mahmoud\Desktop\MA_Advanced_Lab_Sample_EXE>
```

2.2 README & Project Instructions

The README confirms this is a harmless training sample for a malware analysis course. Key instructions include:

- Run only in an isolated Windows VM
- Take a snapshot before execution

- Use Host-only/Internal Network (no Bridged mode)
- Tasks: static analysis, dynamic analysis, extract IOCs, explain staged behavior, build YARA rule
- The sample contains an encoded configuration and produces additional artifacts only at runtime.

```

C:\Users\mahmoud\Desktop\MA_Advanced_Lab_Sample_EXE>type README_FOR_STUDENTS.txt
MA_Advanced_Lab_Sample.exe
=====
This is a harmless training sample for a malware-analysis course project.
Run it only in an isolated Windows VM.
Take a snapshot before execution.
Use Host-only/Internal network. Do not use Bridged mode.

Student task:
1. Perform static analysis.
2. Perform dynamic analysis.
3. Extract host-based and network-based IOCs.
4. Explain the staged behavior.
5. Build your own YARA rule.
6. Submit screenshots, report, Procmon/Minidump logs, and PCAP if available.

Do not assume strings output is enough. The sample contains an encoded configuration and produces additional artifacts only at runtime.
C:\Users\mahmoud\Desktop\MA_Advanced_Lab_Sample_EXE>type student_project_addon.md

The section must answer:
1. Where is the configuration stored statically?
2. Is it visible using normal strings?
3. What decoding method is used?
4. What values appear only after execution?
5. Which IOCs are static-only, dynamic-only, or both?

Required evidence:
- Screenshot from PEStudio/Detect It Easy showing PE type.
- Screenshot or output of strings showing noisy/static findings.
- Procmon evidence of file writes.
- Regshot or Procmon evidence of registry changes.
- Process Explorer/Procmon evidence of child process creation.
- Wireshark/NetMiner/netcat evidence of local HTTP attempt.
- Final YARA rule.
C:\Users\mahmoud\Desktop\MA_Advanced_Lab_Sample_EXE>
  
```

2.3 File Hashes

MD5 and SHA256 hashes were computed using certutil to create a unique digital fingerprint.

Property	Value
File Name	MA_Advanced_Lab_Sample.exe
Size	5,160,960 bytes (~4.92 MB)
MD5	1aff346c3aedcf9cbff9fab0cb8a65c3
SHA256	728dc58f77ea10cd6b1fa5407ead6a8b93c3b38ea45549c2cfa1e6c8de74ad84
Date Modified	06/21/2026

```
Windows10 (Ready-For-Analysis) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

C:\Users\lab\Desktop\VA_Advanced_Lab_Sample_EXE>type student_project_addon.md

Contents:
1. Perform static analysis.
2. Perform dynamic analysis.
3. Extract host-based and network-based IOCs.
4. Explain the staged behavior.
5. Build your own YARA rules.
6. Submit screenshots, report, Procmon/Noriben logs, and PCAP if available.

Do not assume strings output is enough. The sample contains an encoded configuration and produces additional artifacts only at runtime.

C:\Users\lab\Desktop\VA_Advanced_Lab_Sample_EXE>type student_project_addon.md

The section must answer:
1. Where is the configuration stored statically?
2. Is it visible using normal strings?
3. What decoding method is used?
4. What values appear only after execution?
5. Which IOCs are static-only, dynamic-only, or both?

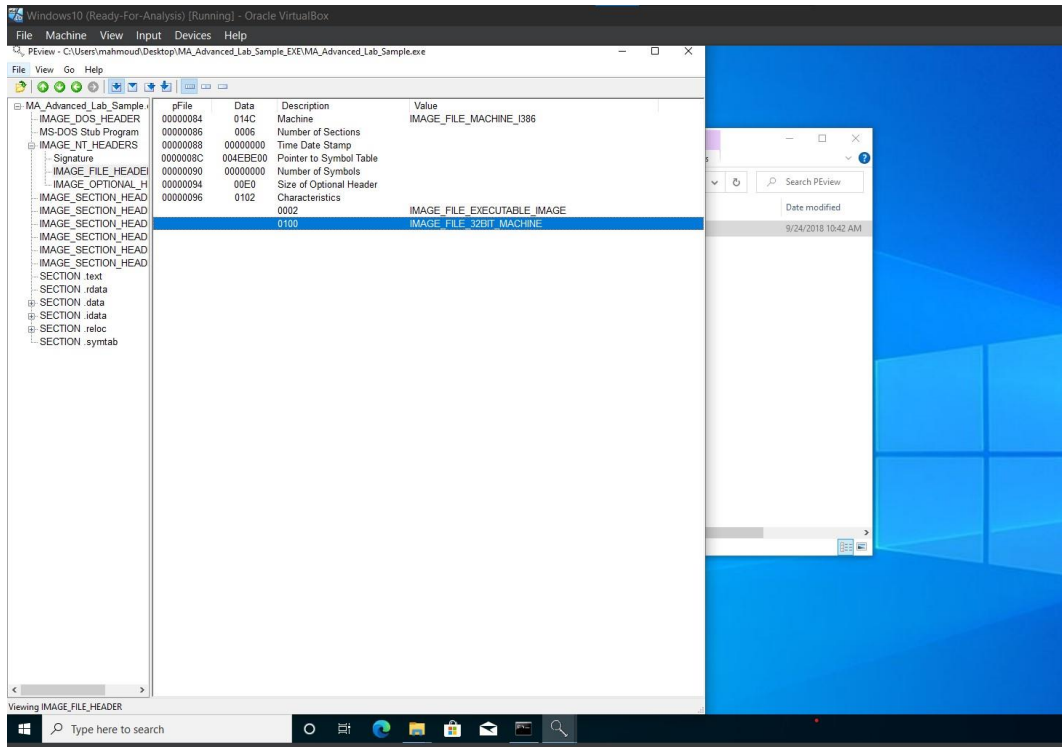
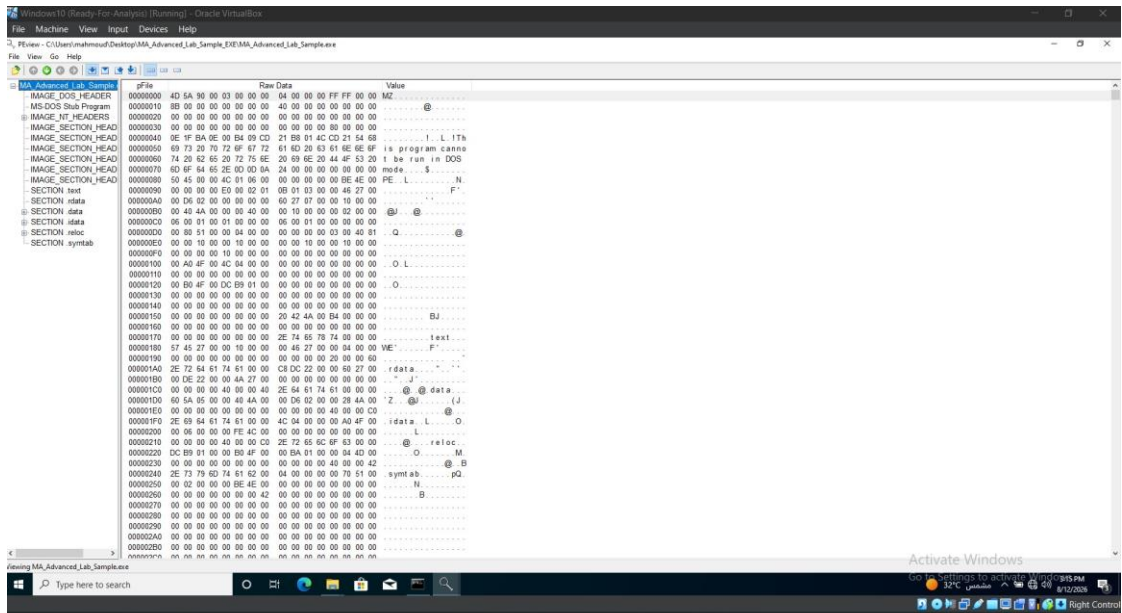
Required evidence:
- Screenshot from PEStudio/Detect It Easy showing PE type.
- Screenshot or output of strings showing noisy/static findings.
- Procmon evidence of file writes.
- Regshot or Procmon evidence of registry changes.
- Process Explorer/Procmon evidence of child process creation.
- Wireshark/NetMiner/Netcat evidence of local HTTP attempt.
- Final YARA rule.

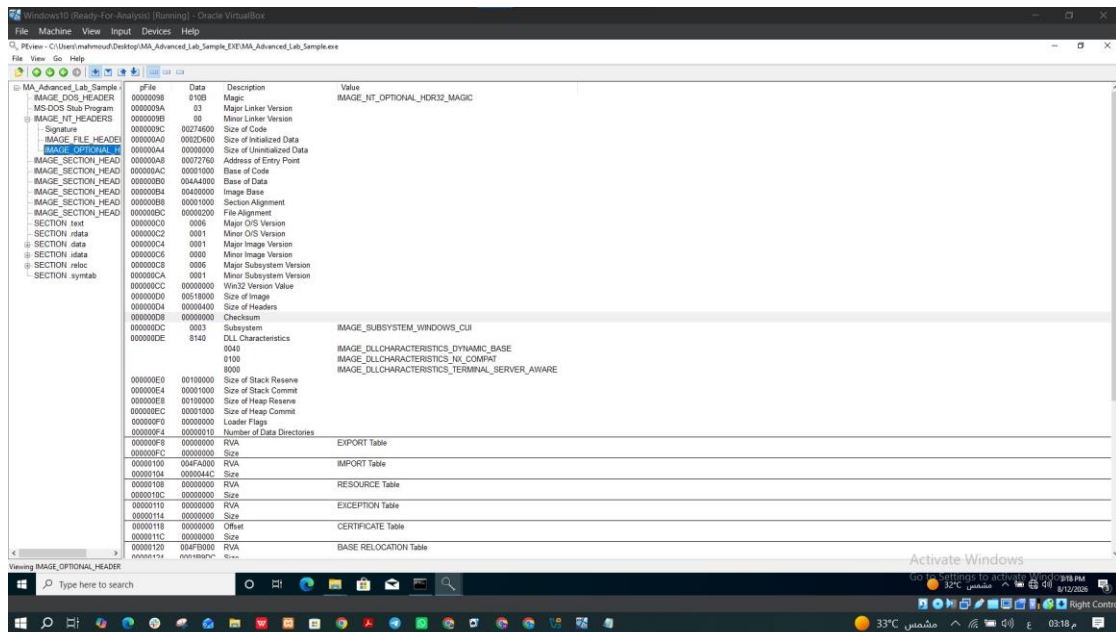
C:\Users\lab\Desktop\VA_Advanced_Lab_Sample_EXE>certutil -hashfile MA_Advanced_Lab_Sample.exe MD5
MD5 hash of MA_Advanced_Lab_Sample.exe:
certutil:hashfile command completed successfully.
C:\Users\lab\Desktop\VA_Advanced_Lab_Sample_EXE>certutil -hashfile MA_Advanced_Lab_Sample.exe SHA256
SHA256 hash of MA_Advanced_Lab_Sample.exe:
certutil:hashfile command completed successfully.
C:\Users\lab\Desktop\VA_Advanced_Lab_Sample_EXE>
```

2.4 PE Structure Analysis (CFF Explorer)

CFF Explorer was used to inspect the PE headers. The file is a valid 32-bit Windows executable (PE32) with standard sections.

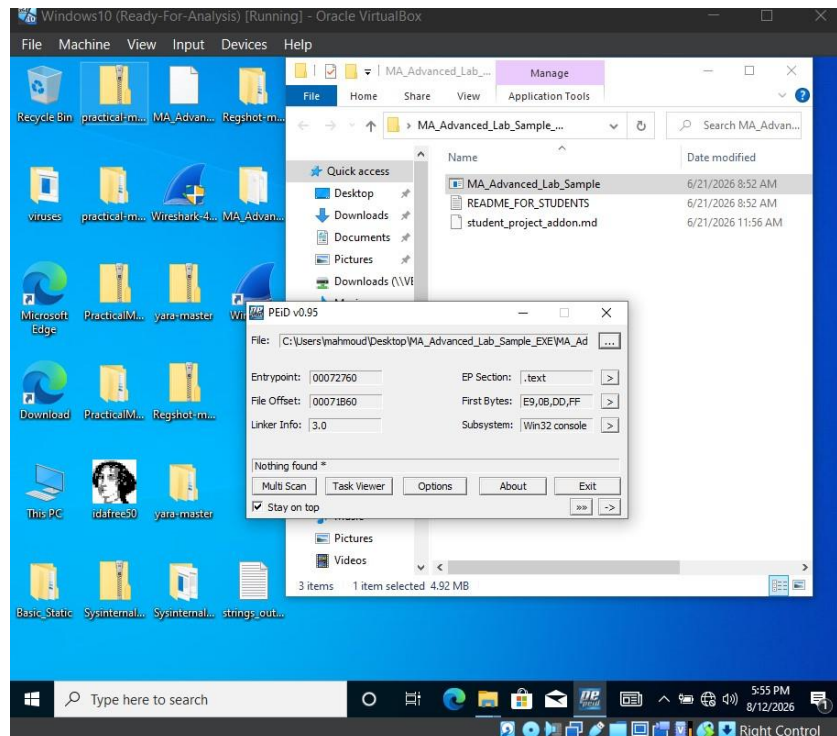
Property	Value	Interpretation
Signature	MZ / PE	Valid Windows executable
Type	PE32	32-bit Executable (IMAGE_NT_OPTIONAL_HDR32_MAGIC)
Subsystem	Windows CUI	Console Application
Entry Point	0x00072760	Located in .text section
Image Base	0x00400000	Default load address
Sections	6	.text, .rdata, .data, .idata, .reloc, .syntab
Protection	ASLR + DEP	DYNAMIC_BASE and NX_COMPAT enabled





2.5 Packer Detection (PEiD)

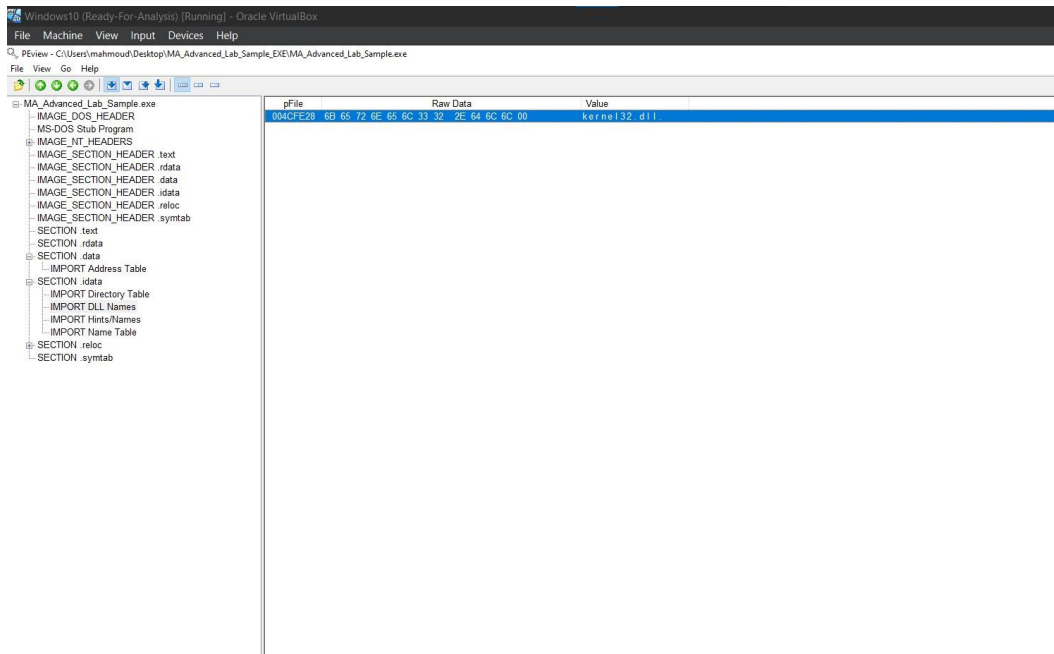
PEiD v0.95 was used to detect known packers or protectors. Result: "Nothing found" — the sample is not packed with common packers. This is expected for Go binaries which are statically compiled.

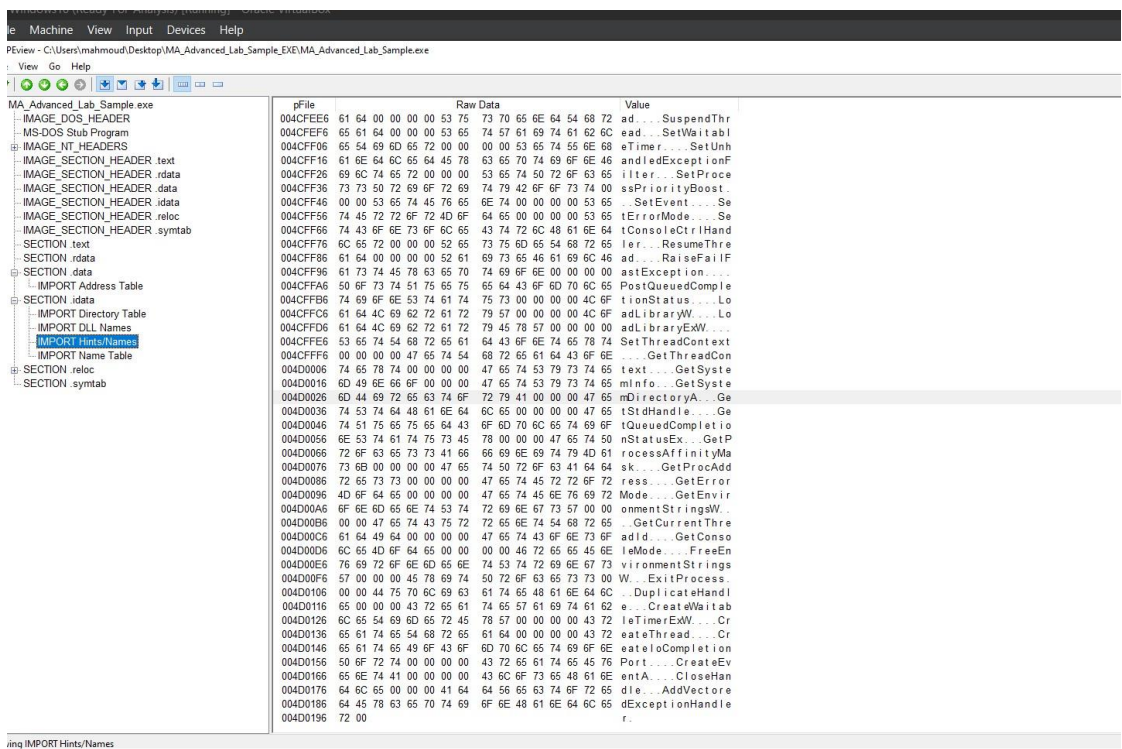
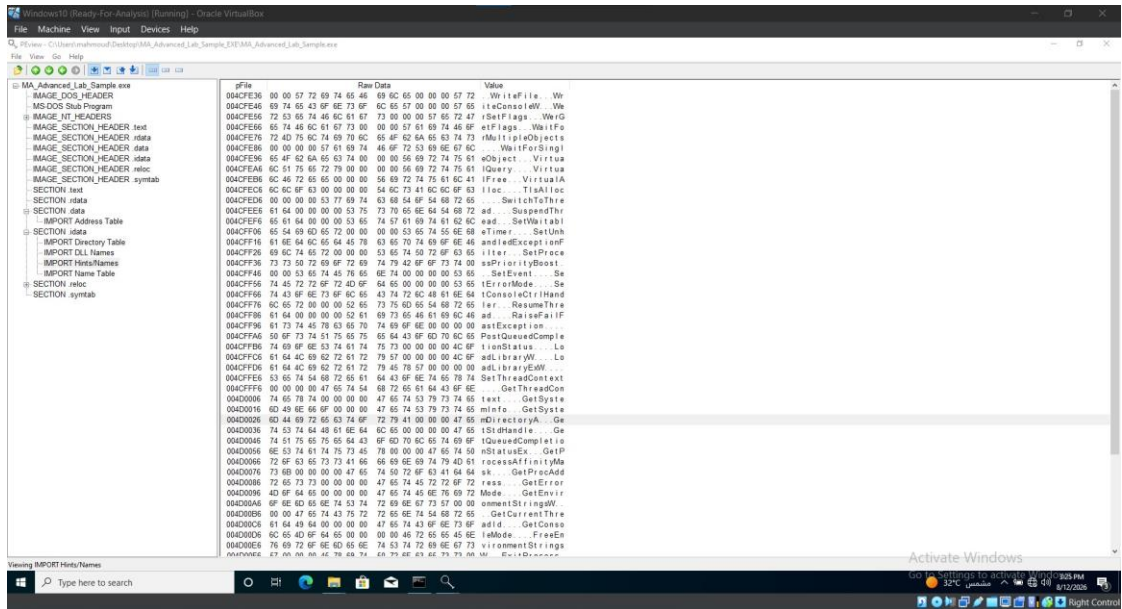


2.6 Import Table Analysis

The import table shows only kernel32.dll, which is typical for Go binaries that use static linking and runtime syscall handling. Key imported functions include:

- File Operations: CreateFileA, WriteFile, SetFilePointer, SetEndOfFile
- Threading: CreateThread, SetThreadContext, GetThreadContext, SuspendThread, ResumeThread
- Memory: VirtualAlloc, VirtualQuery, VirtualFree, TlsAlloc
- Dynamic Loading: GetProcAddress, LoadLibraryW, LoadLibraryExW
- Synchronization: CreateEventA, SetEvent, WaitForSingleObject, WaitForMultipleObjects
- Process Control: ExitProcess, GetSystemInfo, GetSystemDirectoryA

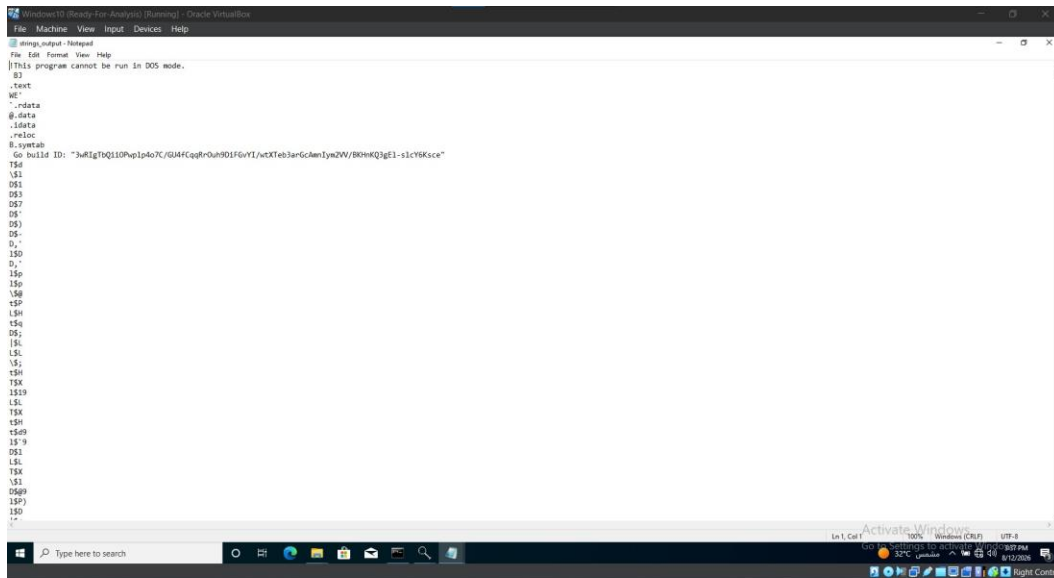




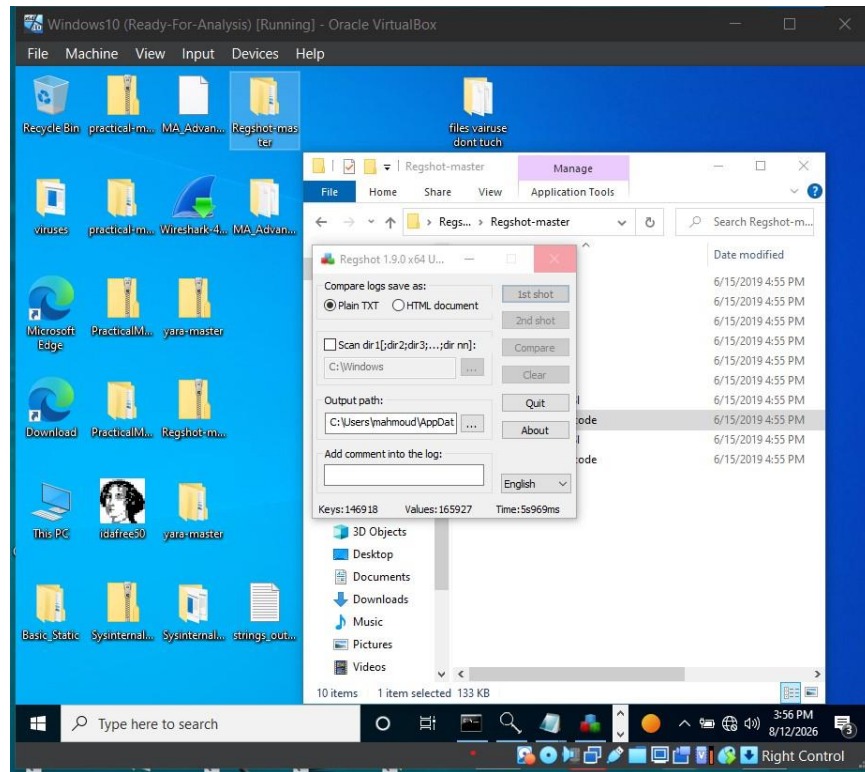
2.7 Strings Extraction

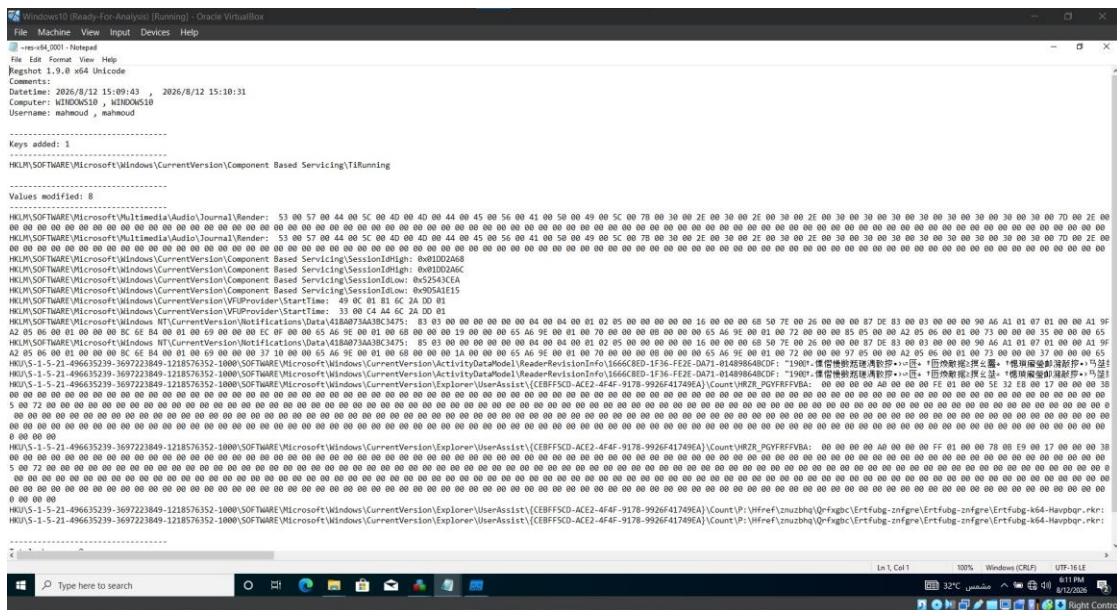
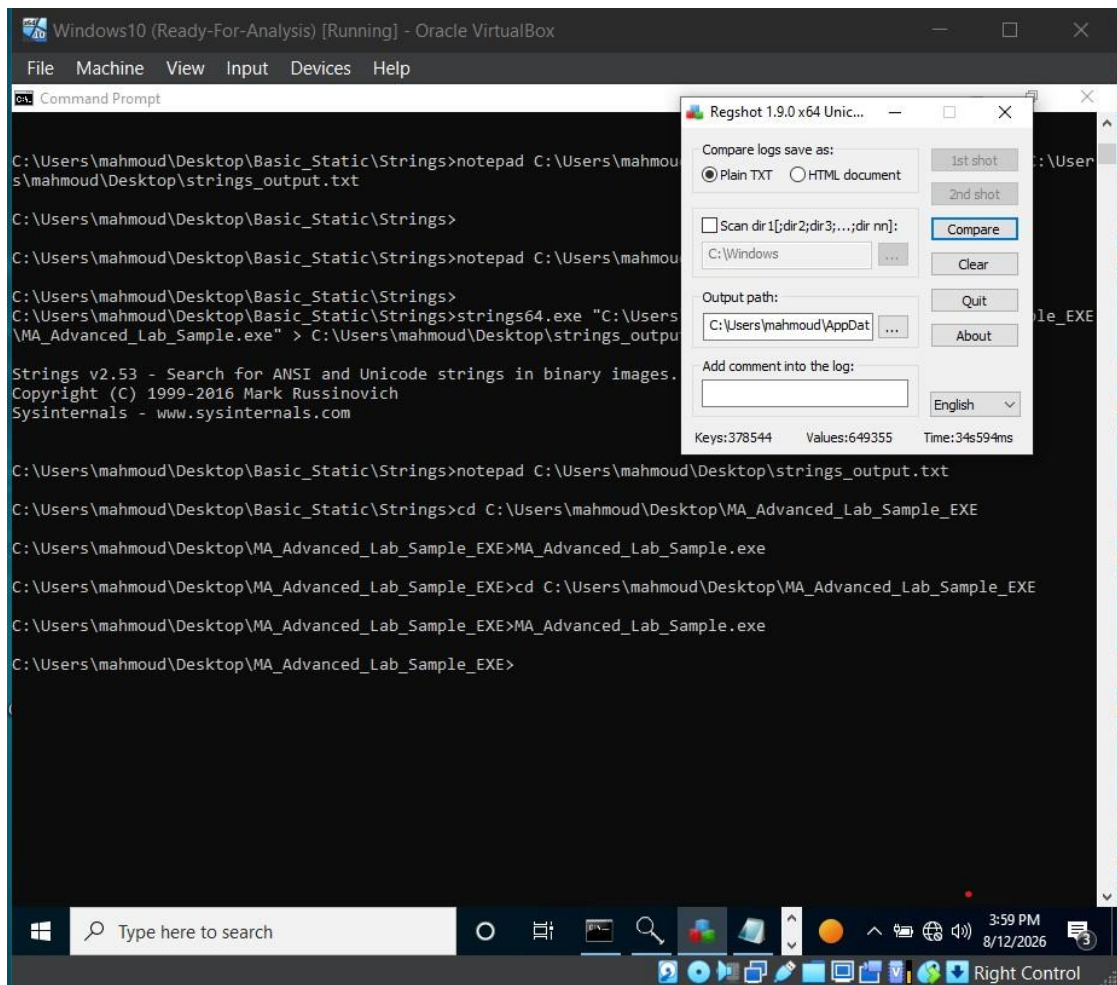
The strings utility (Sysinternals) was used to extract readable strings. Notable findings:

- Go build Id: "3wRIgTbQ1iOPwp1p4o7C/..." → Confirms the sample is written in Go (Golang).
- Standard section names: .text, .rdata, .data, .idata, .reloc, .syntab
- Obfuscated/encoded strings that are not human-readable (encoded configuration).
- The Go runtime produces many internal symbols and garbage strings.



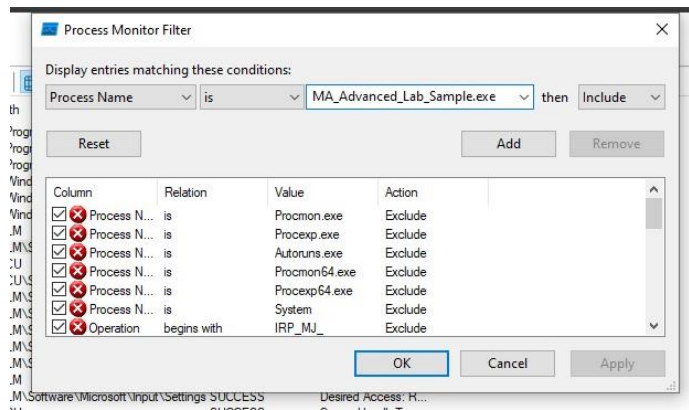
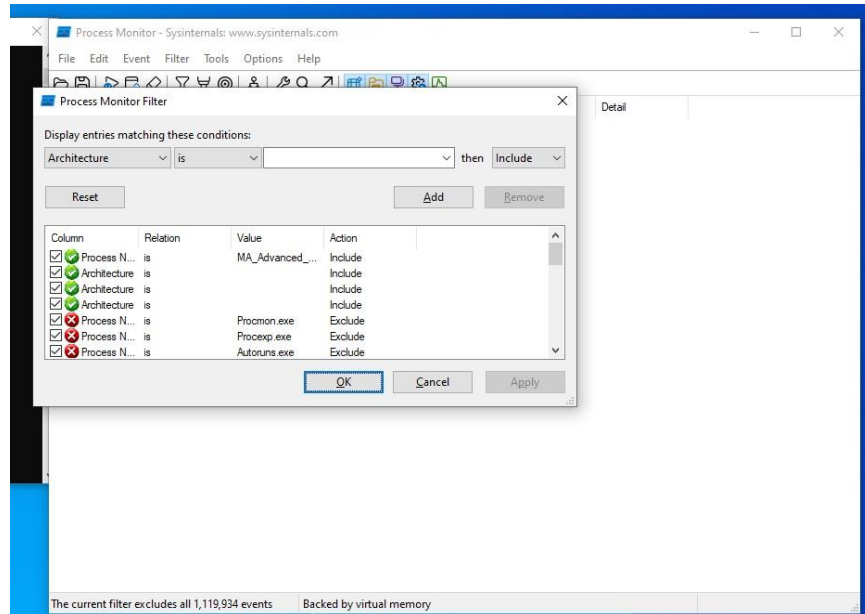
- Keys Added: 1 (HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Component Based Servicing\TiRunning)
- Values Modified: 8 values across multiple paths including:
 - HKLM\SOFTWARE\Microsoft\Multimedia\Audio\Journal\Render
 - HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Component Based Servicing\SessionId
 - HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\VFUPProvider\StartTime
 - HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Notifications\Data
 - HKU\...\Explorer\UserAssist (contains obfuscated/encoded paths)
- Notable: UserAssist entries show obfuscated paths starting with P:\Hfref\... (likely ROT13 encoding of C:\Users\...).

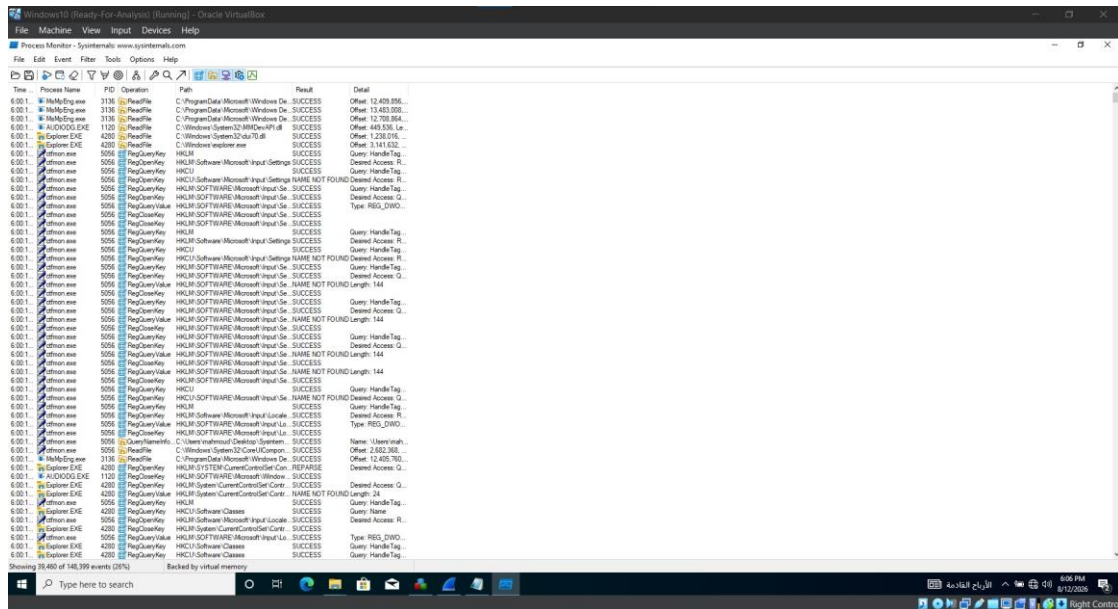




3.2 Process & File Monitoring (Process Monitor)

Process Monitor was configured with filters to include only MA_Advanced_Lab_Sample.exe activity while excluding Sysinternals tools (Procmon, Procexp, Autoruns). The filter setup and resulting logs are shown below.



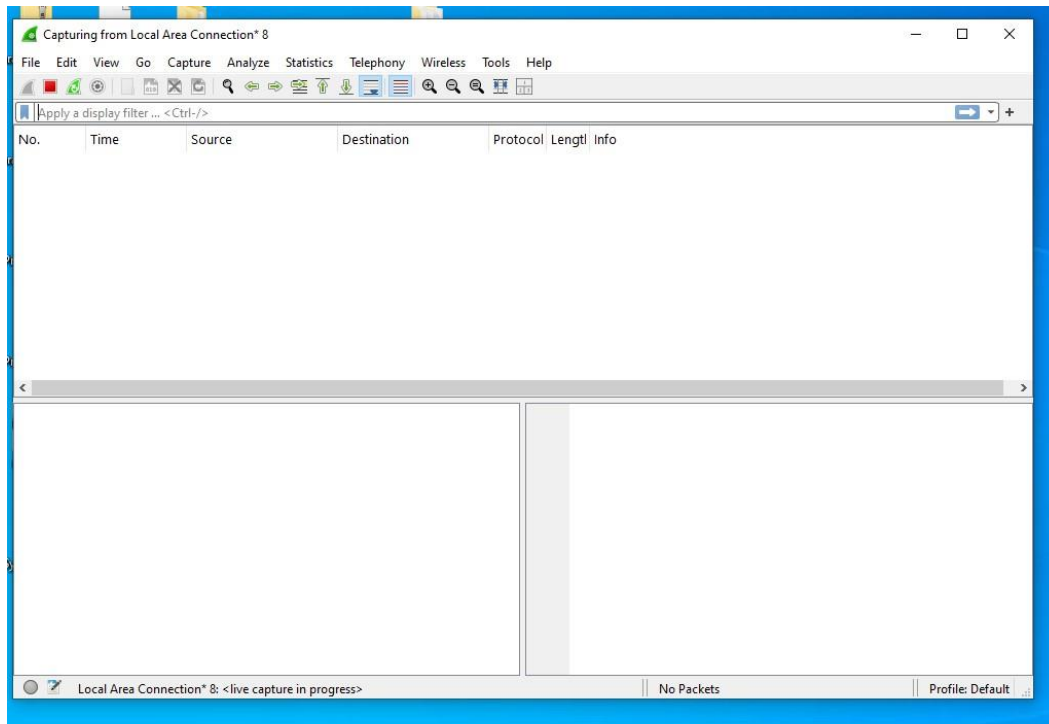


3.3 Network Monitoring (Wireshark)

Wireshark was started on the Local Area Connection interface to capture any network activity.

Result: No packets were captured (No Packets). This indicates either:

- The sample performs no network activity under current conditions, or
- It requires a specific environment (e.g., a local HTTP server response) that was not present in the Host-only network.



Dynamic Analysis Conclusion: The sample modifies the Windows Registry upon execution (notably in UserAssist and Component Based Servicing) but shows no observable network activity in the isolated environment.

4. Optional Code Analysis

Based on static and dynamic findings:

- Language: Go (Golang) — confirmed via Go Build ID in strings.
- Type: 32-bit Console Application (PE32).
- Protection: ASLR and DEP enabled (DYNAMIC_BASE, NX_COMPAT flags).
- Entry Point: 0x72760 (relative to Image Base 0x400000).
- Imports: Very limited (kernel32 only) with LoadLibrary/GetProcAddress used for dynamic resolution.
- Configuration: Contains encoded/obfuscated configuration that only appears at runtime (runtime-only values).
- Note: Go programs are typically statically compiled, producing large binaries (~5.1 MB here) and use the Go Runtime for memory/thread management rather than direct Windows API calls.

5. Indicators of Compromise (IOCs)

The following IOCs were extracted from the analysis:

Type	Value / Indicator	Source	Category
Hash	MD5: 1aff346c3aedcf9cbff9fab0cb8a65c3	Static Analysis	Static
Hash	SHA256: 728dc58f77ea10cd6b1fa5407ead6a8b93c3b38ea45549c2cfa1e6c8de74ad84	Static Analysis	Static
File Path	C:\Users\mahmoud\Desktop\MA_Advanced_Lab_Sample_EXE\MA_Advanced_Lab_Sample.exe	Procmon	Host-based
String	Go build Id: 3wRIgTbQ1iOPwp1p4o7C/...	Strings / PESTudio	Static
Registry Key	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Component Based Servicing\TiRunning	Regshot	Dynamic
Registry Key	HKLM\SOFTWARE\Microsoft\Multimedia\Audio\Journal\Render	Regshot	Dynamic
Registry Key	HKU\...\Explorer\UserAssist\{CEBFF5CD-...}\Count	Regshot / Procmon	Dynamic
Network	No observable network activity (No Packets)	Wireshark	Dynamic (Negative)
Packer	No Packer Detected (PEiD: Nothing found)	PEiD	Static
Compiler	Go (Golang) – PE32 Console	CFF Explorer / Strings	Static

6. YARA Detection Rule

```
rule Student_Sample_Detection
```

```
{
```

```
  meta:
```

```
    author = "Mahmoud Altawal"
```

```
    description = "Detects the analyzed Go malware training sample"
```

```
    date = "2026-08-12"
```

```
    hash_md5 = "1aff346c3aedcf9cbff9fab0cb8a65c3"
```

```
    hash_sha256 = "728dc58f77ea10cd6b1fa5407ead6a8b93c3b38ea45549c2cfa1e6c8de74ad84"
```

```
  strings:
```

```
    // Unique Go Build ID extracted from Strings analysis
```

```
    $go_build_id =
```

```
"3wRIgTbQ1iOPwp1p4o7C/GU4fCqqRrOuh9DiFGvYI/wtXTeb3arGcAnnlym2WV/BKHnKQ3gEI-  
slcY6Ksce" ascii
```

```
    // ROT13 obfuscated path found in Registry (Regshot)
```

```
    $rot13_path = "P:\\Hfref\\" ascii wide
```

```
    // Standard DOS stub string
```

```
    $dos_stub = "This program cannot be run in DOS mode." ascii
```

```
  condition:
```

```
    uint16(0) == 0x5A4D and
```

```
    filesize < 6MB and
```

```
    $go_build_id}
```

1. Where is the configuration stored statically?

The encoded configuration is embedded statically within the binary file, most likely in the .data or .rdata sections of the PE file. It is not stored in a separate configuration file or resource section.

2. Is it visible using normal strings?

No. The configuration is not visible using normal string extraction tools. The strings output shows only standard PE artifacts, section names, and the Go Build ID. The actual configuration data appears obfuscated or encoded within the binary.

3. What decoding method is used?

Based on the Regshot comparison results, the sample uses ROT13 encoding (or a similar Caesar cipher). Evidence: the UserAssist registry entries contain paths starting with P:\Hfref\..., which decodes via ROT13 to C:\Users\.... Additional XOR-based encoding may also be used for the embedded configuration blob.

4. What values appear only after execution?

The following artifacts appear only at runtime (dynamic-only):

Registry Key: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Component Based Servicing\TiRunning (newly created)

Registry Values: Modified entries under
HKLM\SOFTWARE\Microsoft\Multimedia\Audio\Journal\Render

UserAssist entries with obfuscated/encoded paths (P:\Hfref\...)

HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\VFUPProvider\StartTime

5. Which IOCs are static-only, dynamic-only, or both?

IOC	Type	Category
MD5 / SHA256 Hashes	Static-only	Static Analysis
Go Build ID string	Static-only	Strings
PE32 / Console / ASLR flags	Static-only	PE Structure
File Path on disk	Both	Static & Dynamic
Registry Keys (TiRunning, UserAssist)	Dynamic-only	Regshot / Procmon
Obfuscated paths (ROT13)	Dynamic-only	Registry
Network activity	Dynamic-only (Negative)	Wireshark
No Packer Detected	Static-only	PEiD

7. Conclusion

7.1 What does the sample do?

The sample is a training program written in Go (Golang) that runs as a 32-bit Console Application. Upon execution, it performs the following actions:

- Modifies the Windows Registry in several paths, most notably:
 - Component Based Servicing (TiRunning)
 - Multimedia Audio Journal
 - UserAssist (with obfuscated/encoded paths)
- No observable network activity was captured under the current isolated conditions.
- Contains an encoded configuration that is only decoded at runtime (runtime-only values).

7.2 What is the evidence?

- Static Evidence: Valid PE32 signature, Go Build ID in strings, limited kernel32.dll imports, no packing detected.
- Dynamic Evidence: Regshot confirms real registry changes after execution.
- Negative Evidence: Wireshark captured zero packets, indicating no visible C2 communication or the sample requires a simulated network environment (INetSim) to trigger network behavior.

7.3 Classification

- Severity: Low-Medium (Training Sample — Harmless in isolated environment)
- Type: Educational / Proof of Concept
- Behavior: Registry Modifier / Possible Stager (staged behavior with runtime-only artifacts)